

KLASIFIKASI DATASET ECOLI MENGUNAKAN K-NEAREST NEIGHBORS

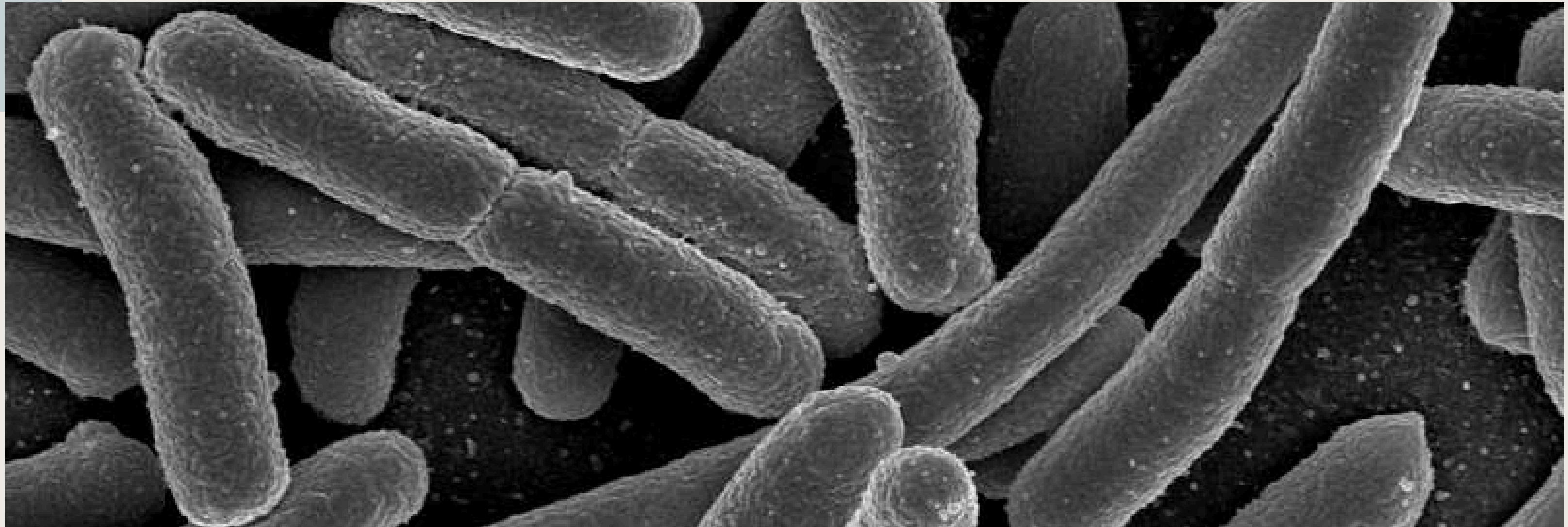
PENGENALAN KECERDASAN BUATAN IF 46-09

KELOMPOK 13

Sherly Angelina (1301223370)

Nur afny putri pratiwi is (1301223256)

M salman razak alfarisi (1301220487)



PENDAHULUAN

Dataset yang digunakan adalah dataset Ecoli, yang digunakan untuk melakukan klasifikasi jenis lokasi subcellular protein pada dataset bakteri Ecoli.

Dataset ini terdiri dari 2 bagian utama:

- dataset_feature
- dataset_target

1



	mcg	gvh	lip	chg	aac	alm1	alm2
0	0.49	0.29	0.48	0.5	0.56	0.24	0.35
1	0.07	0.40	0.48	0.5	0.54	0.35	0.44
2	0.56	0.40	0.48	0.5	0.49	0.37	0.46
3	0.59	0.49	0.48	0.5	0.52	0.45	0.36
4	0.23	0.32	0.48	0.5	0.55	0.25	0.35
...	...	...	...	...	...	...	...
331	0.74	0.56	0.48	0.5	0.47	0.68	0.30
332	0.71	0.57	0.48	0.5	0.48	0.35	0.32
333	0.61	0.60	0.48	0.5	0.44	0.39	0.38
334	0.59	0.61	0.48	0.5	0.42	0.42	0.37
335	0.74	0.74	0.48	0.5	0.31	0.53	0.52

336 rows × 7 columns



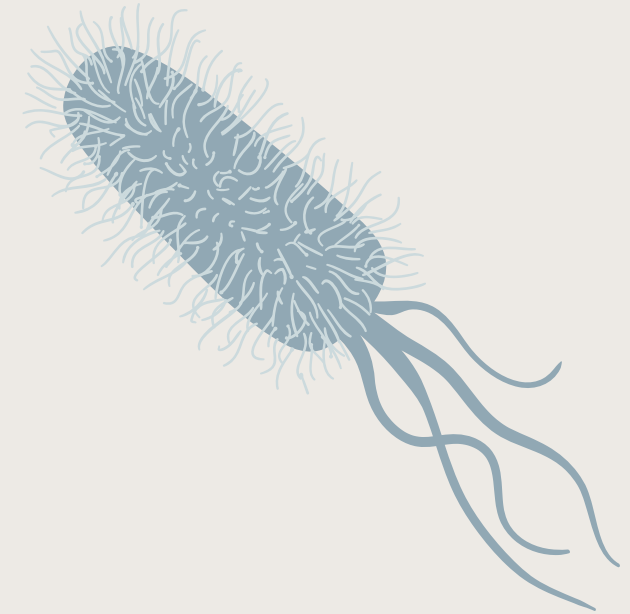
2

	class
0	cp
1	cp
2	cp
3	cp
4	cp
...	...
331	pp
332	pp
333	pp
334	pp
335	pp

336 rows × 1 columns

KETERANGAN FITUR

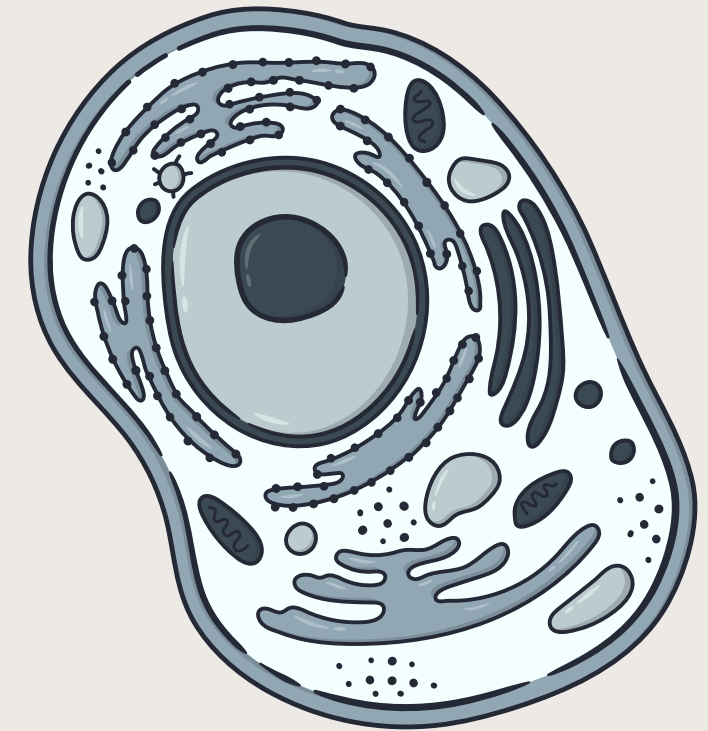
- 1 **mcg** : metode McGeoch untuk pengenalan urutan sinyal.
- 2 **gvh** : metode Von Heijne untuk pengenalan urutan sinyal.
- 3 **lip** : skor konsensus urutan sinyal Lipoprotein Von Heijne.
- 4 **chg** : keberadaan muatan pada N-terminus dari lipoprotein yang diprediksi.
- 5 **aac** : skor dari analisis diskriminan dari isi asam amino dari protein membran luar dan periplasmik.
- 6 **alm1** : skor dari program prediksi daerah melintang membran ALOM.
- 7 **alm2** : skor dari program ALOM setelah mengeluarkan daerah sinyal yang dapat dipotong dari urutan.



KETERANGAN TARGET

Lokasi subcelullar protein yang akan di prediksi

- 1 **cp** : sitoplasma
- 2 **im** : membran bagian dalam tanpa urutan sinyal
- 3 **pp** : periplasma
- 4 **imU** : membran bagian dalam, urutan sinyal tidak dapat diubah
- 5 **om** : membran luar
- 6 **omL** : lipoprotein membran luar
- 7 **imL** : lipoprotein membran dalam
- 8 **imS** : membran bagian dalam, urutan sinyal yang dapat dibelah





MASALAH & TUJUAN

- Masalah yang akan diselesaikan adalah melakukan klasifikasi jenis lokasi subcellular protein pada bakteri Ecoli berdasarkan beberapa fitur yang diberikan, dengan menggunakan model klasifikasi yang sesuai.
- Tujuan utamanya adalah untuk memprediksi letak/lokasi subcelular protein dari fitur yang diberikan dengan akurasi sebaik mungkin.



DATA PREPROCESSING

Tujuan dari preprocessing data adalah untuk mempersiapkan data mentah agar siap digunakan untuk analisis atau pemodelan learning. Dengan melakukan preprocessing data yang tepat, kita dapat meningkatkan kualitas dan akurasi dari hasil analisis atau pemodelan yang dilakukan.



1

24/7 Accessibility

2

Hybrid Cloud Service

3

Data Loss Prevention

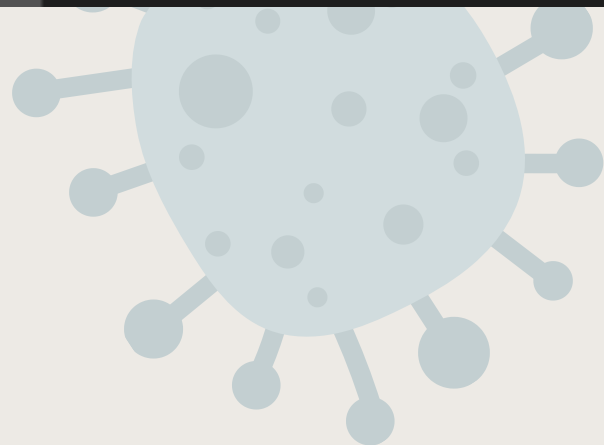
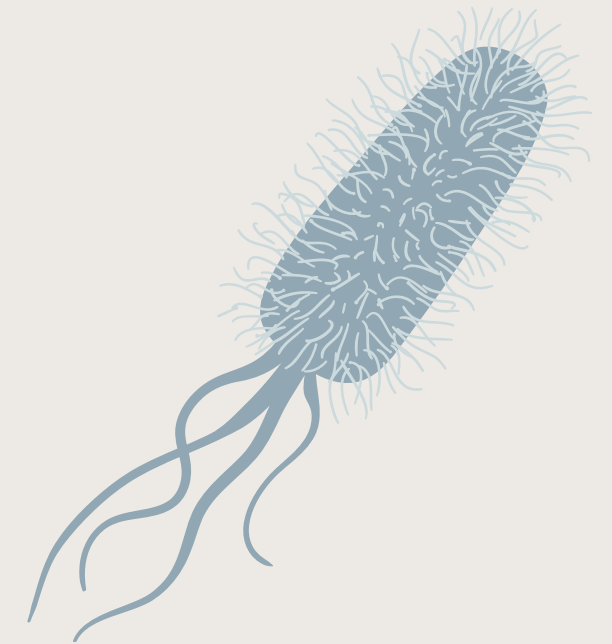


INFORMASI DATASET

```
1 # Cek Info Data
2 print("Info data fitur")
3 dataset_feature.info()
4
5 print("\nInfo data target")
6 dataset_target.info()
```

```
Info data fitur
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 336 entries, 0 to 335
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    mcg     336 non-null    float64
 1    gvh     336 non-null    float64
 2    lip     336 non-null    float64
 3    chg     336 non-null    float64
 4    aac     336 non-null    float64
 5    alm1    336 non-null    float64
 6    alm2    336 non-null    float64
dtypes: float64(7)
memory usage: 18.5 KB
```

```
Info data target
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 336 entries, 0 to 335
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    class   336 non-null    object
dtypes: object(1)
memory usage: 2.8+ KB
```





PENCARIAN NILAI YANG HILANG

```
1 # Cek Missing Value
2 print("Missing Value data fitur")
3 print(dataset_feature.isnull().sum())
4
5 print("\nMissing Value data target")
6 print(dataset_target.isnull().sum())
```

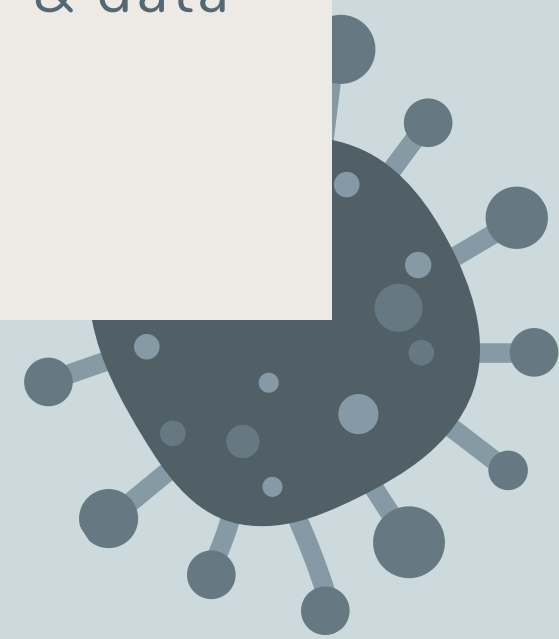
Missing Value data fitur

```
mcg      0
gvh      0
lip      0
chg      0
aac      0
alm1     0
alm2     0
dtype: int64
```

Missing Value data target

```
class    0
dtype: int64
```

Hasil pencarian Missing Value pada dataset menunjukkan bahwa tidak ada nilai yang hilang pada data fitur & data target





PERSEBARAN DATA TARGET

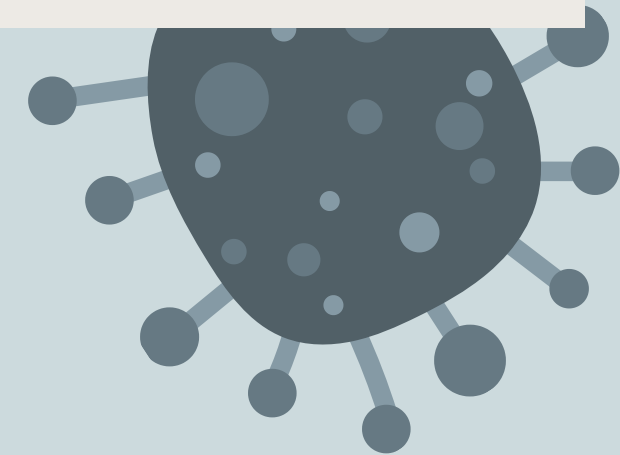
```
1 # Cek Persebaran Data Target
2 class_counts = dataset_target['class'].value_counts()
3 print("Jumlah sampel dalam setiap kelas:")
4 print(class_counts)
5
6 class_proportions = class_counts / class_counts.sum()
7 print("\nProporsi setiap kelas:")
8 print(class_proportions)
```

Proporsi setiap kelas dihitung dengan membagi jumlah kemunculan kelas tersebut dengan total jumlah data dalam dataset.

Digunakan untuk memeriksa seimbang atau tidaknya dataset dalam hal distribusi kelas.

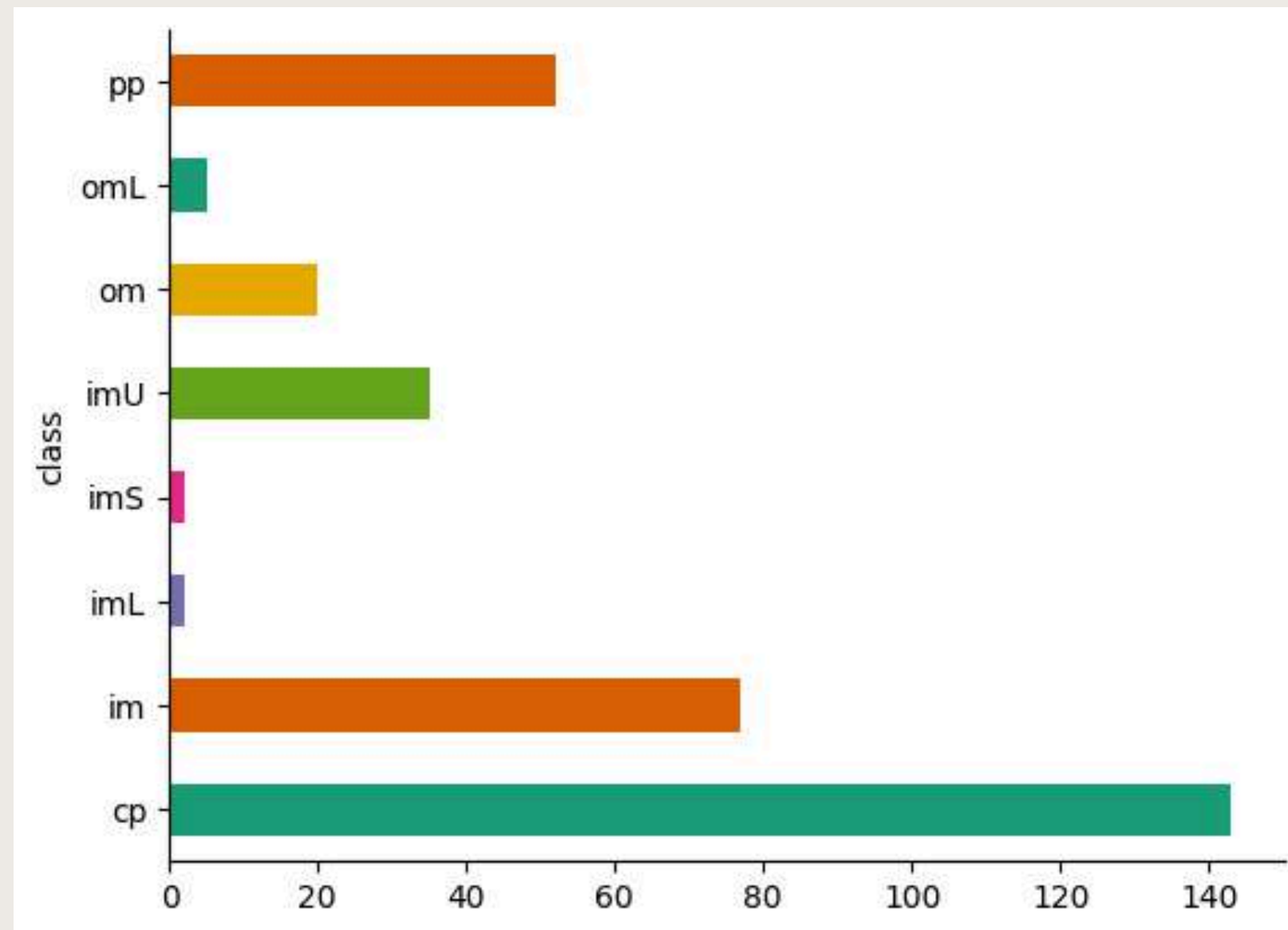
```
➡ Jumlah sampel dalam setiap kelas:
class
cp      143
im      77
pp      52
imU     35
om      20
omL      5
imS      2
imL      2
Name: count, dtype: int64
```

```
Proporsi setiap kelas:
class
cp      0.425595
im      0.229167
pp      0.154762
imU     0.104167
om      0.059524
omL     0.014881
imS     0.005952
imL     0.005952
Name: count, dtype: float64
```





VISUALISASI DATA TARGET

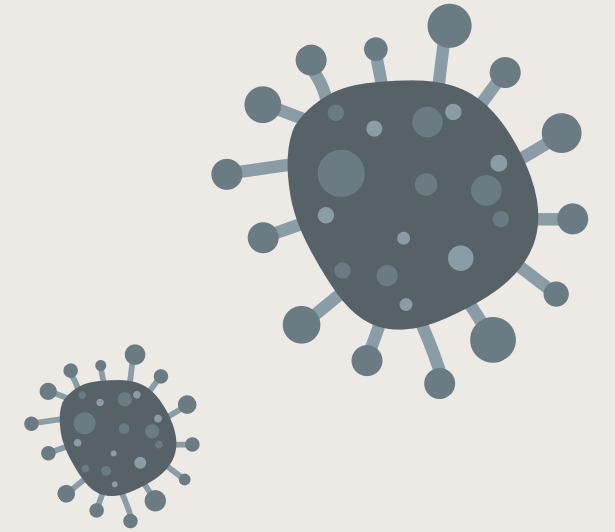


Bar plot tersebut merupakan panjang batang pada setiap kategori yang menunjukkan nilai atau skor class (kategori) pada skala sumbu X.

Kesimpulan yang didapat, class (kategori) cp memiliki nilai paling besar dan class (kategori) imL & imS memiliki nilai paling kecil.



CONCAT DATA



```
1 dataset_ecoli = pd.concat([dataset_feature, dataset_target], axis=1)
2 dataset_ecoli
```

	mcg	gvh	lip	chg	aac	alm1	alm2	class
0	0.49	0.29	0.48	0.5	0.56	0.24	0.35	0
1	0.07	0.40	0.48	0.5	0.54	0.35	0.44	0
2	0.56	0.40	0.48	0.5	0.49	0.37	0.46	0
3	0.59	0.49	0.48	0.5	0.52	0.45	0.36	0
4	0.23	0.32	0.48	0.5	0.55	0.25	0.35	0
...	...	...	...	...	...	...	...	...
331	0.74	0.56	0.48	0.5	0.47	0.68	0.30	7
332	0.71	0.57	0.48	0.5	0.48	0.35	0.32	7
333	0.61	0.60	0.48	0.5	0.44	0.39	0.38	7
334	0.59	0.61	0.48	0.5	0.42	0.42	0.37	7
335	0.74	0.74	0.48	0.5	0.31	0.53	0.52	7

336 rows x 8 columns

Dataset tersebut terdapat 336 baris & 8 kolom

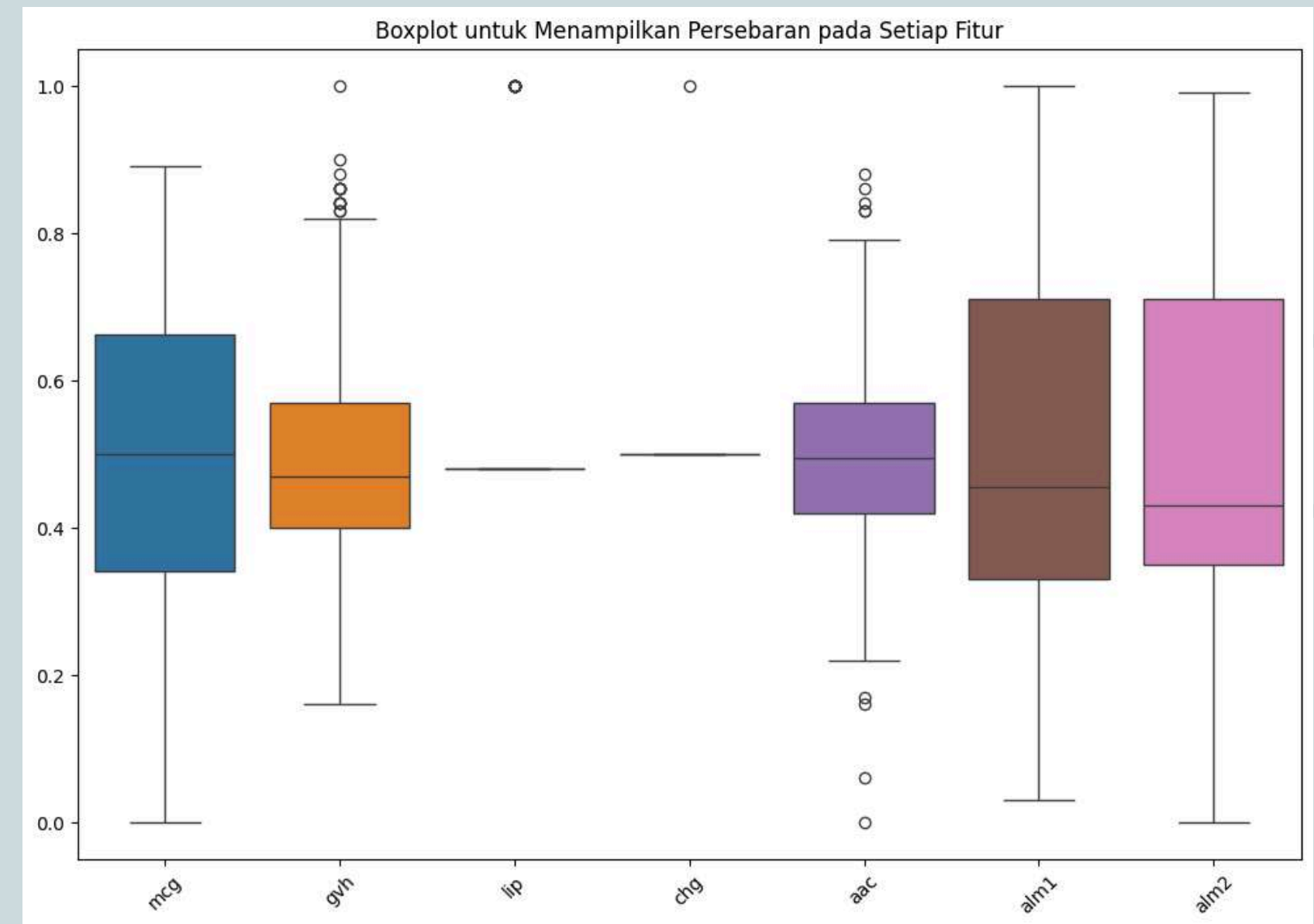
Pada kolom 'class' telah berhasil diencode.



VISUALISASI DATA FITUR

Berdasarkan boxplot yang ditampilkan, adanya perbedaan rentang nilai pada fitur tersebut.

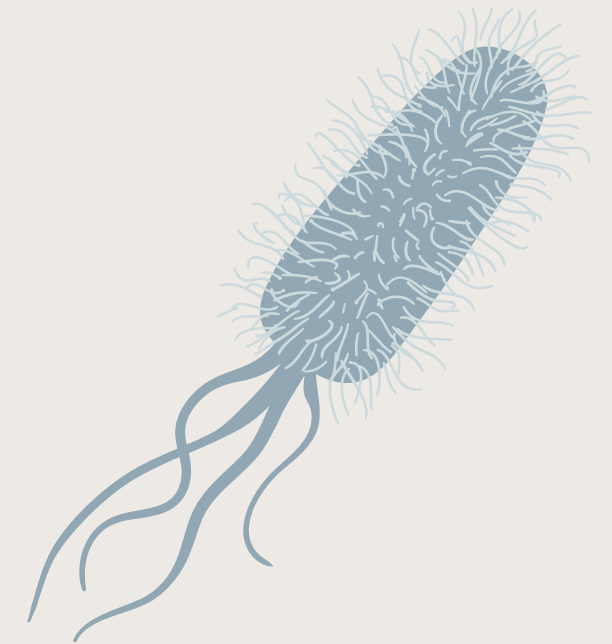
Normalisasi data diperlukan karena dalam pemodelan algoritma kNN, jarak antara sampel data dan tetangga terdekatnya dihitung menggunakan metrik jarak seperti Euclidean. Dimana jika tidak di normalisasi, pemodelan algoritma kNN akan menjadi cenderung terhadap fitur dengan rentang nilai yang lebih besar, sehingga hasil klasifikasi mungkin tidak akurat.





SPLIT DATA & NORMALISASI

Tujuan melakukan split data train dan test sebelum normalisasi adalah untuk menghindari kebocoran informasi dari data uji ke data latih dan menghindari overfitting, dimana model akan memiliki performa yang baik pada data latih, tetapi buruk pada data uji akibat informasi dari data uji bocor selama proses normalisasi.



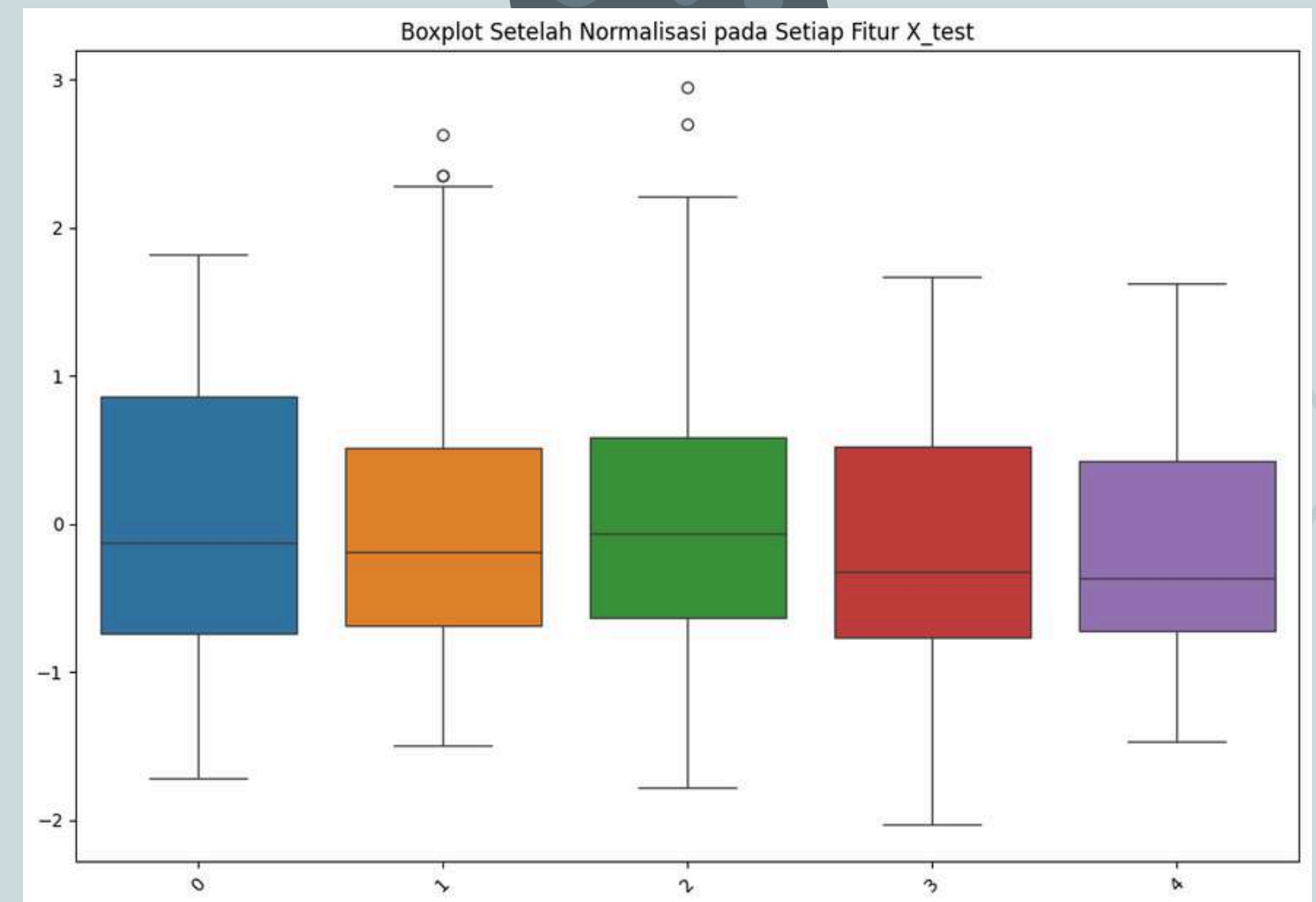
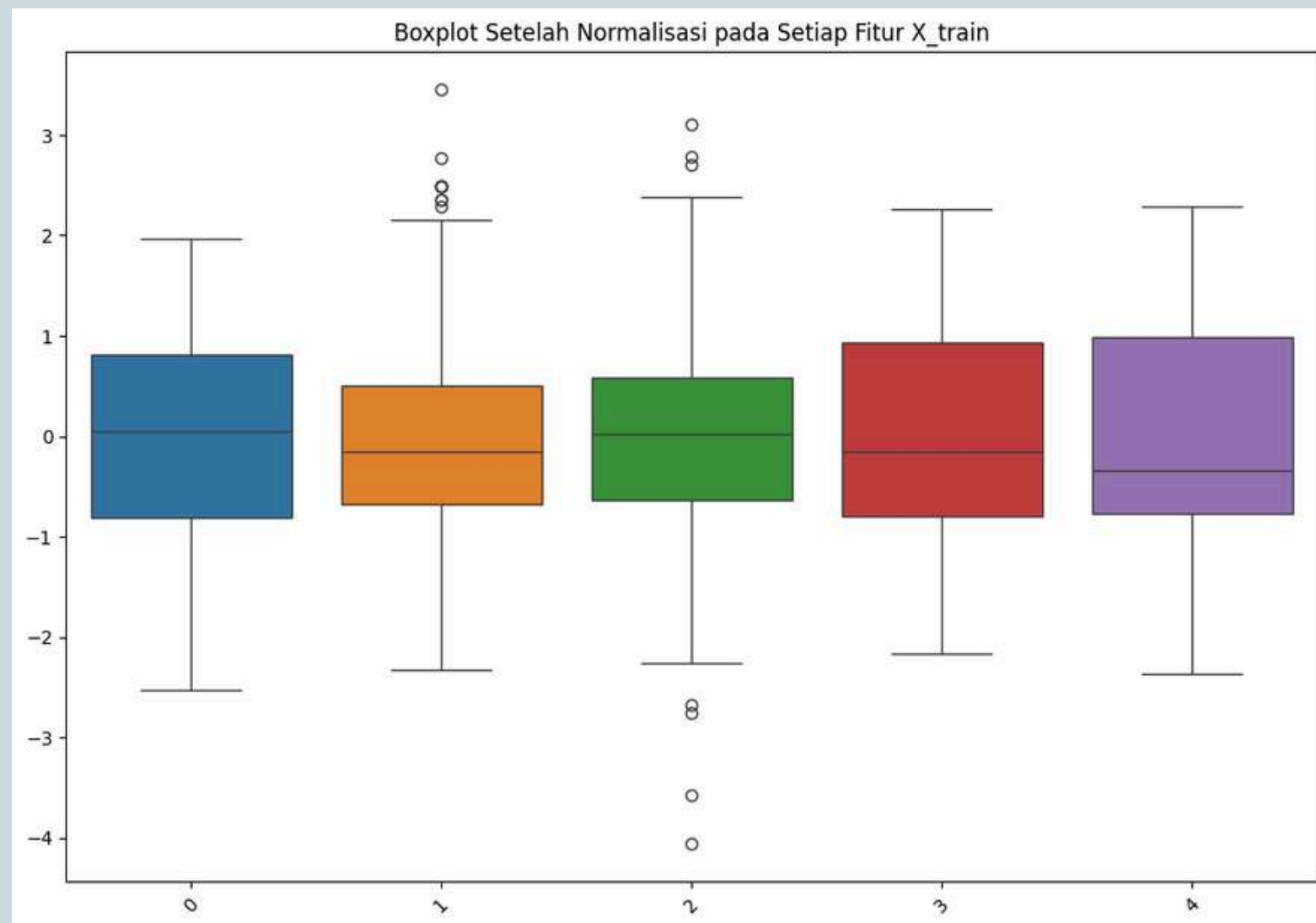
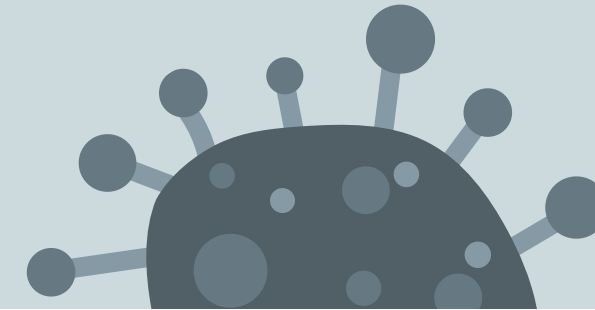
```
1 from sklearn.model_selection import train_test_split
2 # Pisahkan data
3 X = dataset_ecoli.drop(['lip', 'chg', 'class'], axis=1)
4 y = dataset_ecoli['class']
5
6 # Bagi menjadi data uji dan data test
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```



```
1 from sklearn.preprocessing import StandardScaler
2 # Normalisasi data dengan menggunakan StandarScaler
3 scaler = StandardScaler()
4 X_train = scaler.fit_transform(X_train)
5 X_test = scaler.transform(X_test)
```

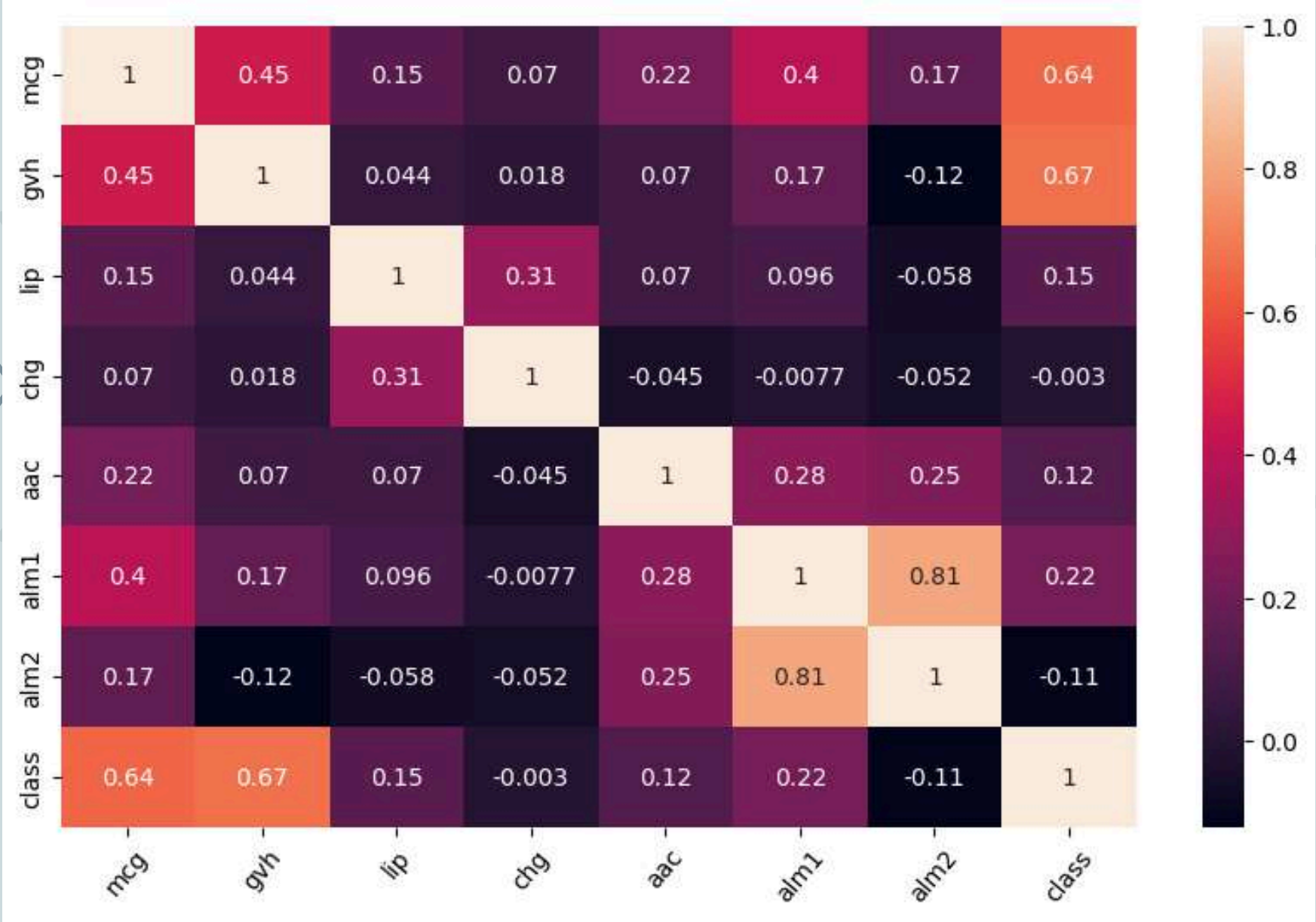


VISUALISASI DATA FITUR





VISUALISASI HEATMAP DATA ECOLI

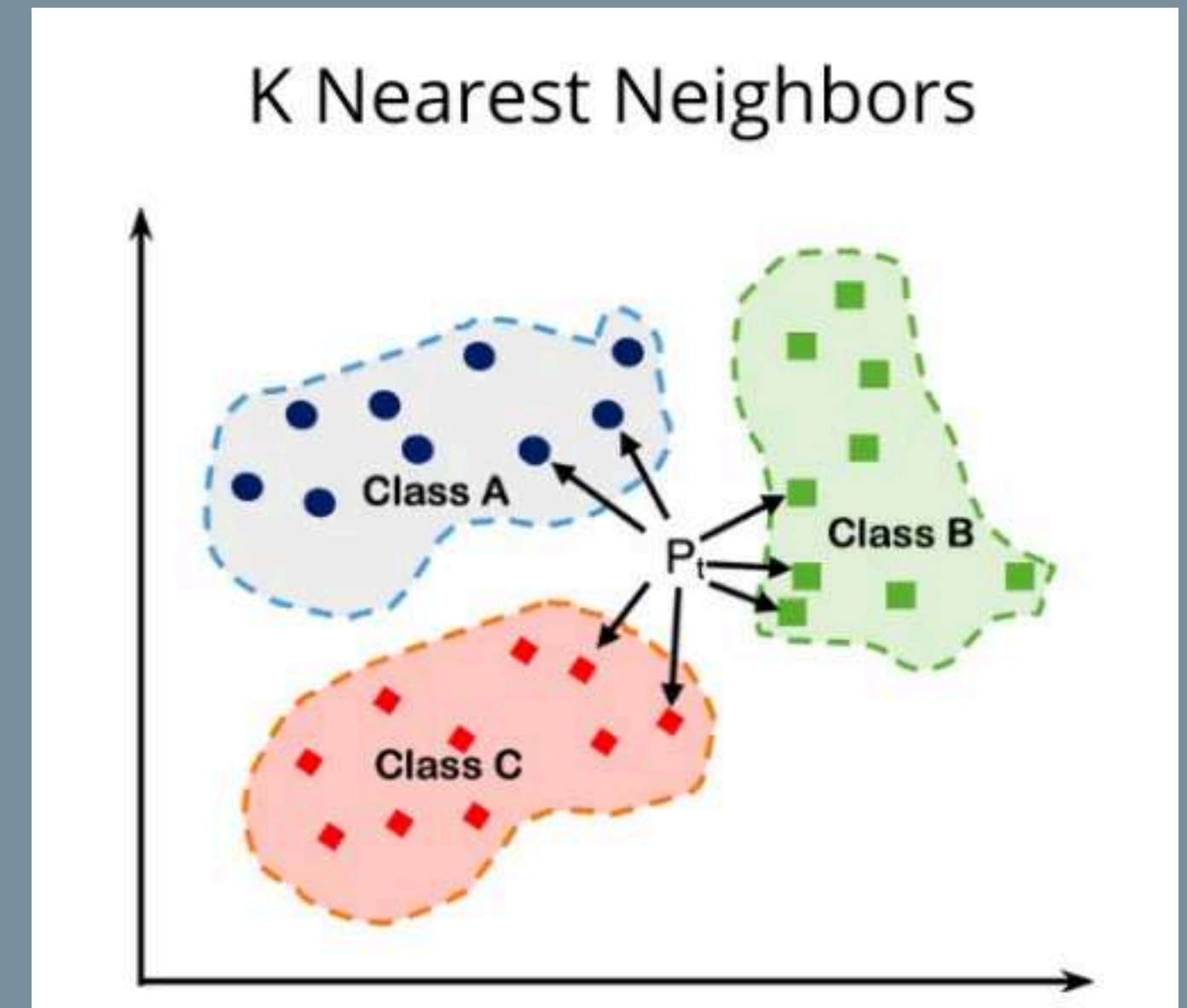




METODE LEARNING

Klasifikasi data Ecoli dengan menggunakan algoritma K-Nearest Neighbors (kNN). kNN merupakan sebuah algoritma dalam machine learning, dimana prediksi dilakukan dengan membandingkan data yang akan diprediksi dengan sejumlah k data tetangga terdekat dalam ruang fitur. Nilai dari k adalah jumlah tetangga terdekat yang dipilih oleh pengguna

- 1 Experiment Settings
- 2 Model Training
- 3 Model Testing & Evaluasi
- 4 Hasil & Analisis





EXPERIMENT SETTINGS

Setting parameter pada percobaan ini meliputi nilai k dan distance. k merupakan jumlah tetangga terdekat yang ditentukan untuk melakukan prediksi dan distance merupakan metrik jarak untuk mengukur jarak antara titik-titik dalam ruang fitur. Nilai default k yang ditentukan adalah 5, namun dalam proses selanjutnya akan ditentukan nilai k yang lain untuk mencari performa terbaik model. Distance pada percobaan ini menggunakan metrik euclidean distance sebagai pengukur jarak antara titik data uji dan setiap titik dalam data latih. Jarak euclidean ini kemudian digunakan untuk menentukan tetangga terdekat dan memprediksi label data uji.

Klasifikasi Data

```
1 # Fungsi untuk menentukan elemen yang paling umum dalam data
2 def most_common(elements):
3     result_most_common = max(set(elements), key=elements.count) # Menghitung elemen yang paling sering muncul
4     return result_most_common # Mengembalikan result_most_common
5
6 # Fungsi untuk menghitung jarak euclidean antara dua titik
7 def euclidean_distance(point1, point2):
8     result_euclidean_distance = np.sqrt(np.sum((point1 - point2)**2, axis=1)) # Menghitung jarak Euclidean antara point1 dan setiap baris dalam point2
9     return result_euclidean_distance # Mengembalikan result_euclidean_distance
```

```
10
11 # Class untuk mendefinisikan Klasifikasi KNN
12 class KNNClassifier:
13     # Fungsi Konstruktor dengan parameter k dan metrik jarak
14     def __init__(self, k=5, distance=euclidean_distance):
15         self.k = k # jumlah tetangga terdekat yang digunakan
16         self.distance = distance # metrik jarak yang digunakan
17
18     # Fungsi untuk melatih model dengan data train
19     def fit(self, X_train, y_train):
20         self.X_train = X_train # Menyimpan data latih
21         self.y_train = y_train # Menyimpan label data latih
22
23     # Fungsi untuk memprediksi label data test
24     def predict(self, X_test):
25         neighbors = [] # Inisialisasi neighbors sebagai list untuk menyimpan tetangga terdekat
26         for x in X_test:
27             distances = self.distance(x, self.X_train) # Menghitung jarak antara x dan setiap titik dalam data latih
28             y_sorted = [y for _, y in sorted(zip(distances, self.y_train))] # Mengurutkan label data latih berdasarkan jarak yang dihitung
29             neighbors.append(y_sorted[:self.k]) # Menyimpan k tetangga terdekat
30         return list(map(most_common, neighbors)) # Mengembalikan label yang paling umum di antara tetangga terdekat
31
32     # Fungsi untuk mengevaluasi performa model berdasarkan data test
33     def evaluate(self, X_test, y_test):
34         y_pred = self.predict(X_test) # Memprediksi label untuk data uji
35         accuracy = sum(y_pred == y_test) / len(y_test) # Inisialisasi accuracy sebagai perhitungan akurasi dengan membandingkan prediksi dengan label sebenarnya
36         return accuracy # Mengembalikan nilai accuracy
```



MODEL TRAINING

```
1 # Mencari k terbaik menggunakan holdout validation
2 accuracies = [] # List untuk menyimpan akurasi dari setiap nilai k yang diuji
3 ks = range(1, 15) # Rentang nilai k yang akan diuji, dari 1 hingga 14
4 best_score = 0.0 # Nilai awal untuk menyimpan akurasi terbaik yang ditemukan
5
6 # Iterasi melalui setiap nilai k yang diuji
7 for k in ks:
8     knn = KNNClassifier(k=k) # Inisialisasi objek KNNClassifier dengan nilai k tertentu
9     knn.fit(X_train, y_train) # Melatih model dengan data latih
10    accuracy = knn.evaluate(X_test, y_test) # Menghitung akurasi model dengan data uji
11    accuracies.append(accuracy) # Menyimpan akurasi untuk nilai k tertentu
12
13    if accuracy > best_score: # Pengondisian untuk memeriksa apakah akurasi yang baru ditemukan lebih baik dari yang sebelumnya terbaik
14        best_score = accuracy # Update nilai akurasi terbaik
15        best_k = k # Simpan nilai k yang menghasilkan akurasi terbaik
16
17    print(f'k = {k}: Akurasi = {accuracy}') # Menampilkan akurasi untuk setiap nilai k
18
19 print(f'k terbaik: {best_k} dengan akurasi: {best_score}') # Mencetak nilai k terbaik beserta akurasinya setelah iterasi selesai
```

```
↔ k = 1: Akurasi = 0.7941176470588235
k = 2: Akurasi = 0.8382352941176471
k = 3: Akurasi = 0.8676470588235294
k = 4: Akurasi = 0.8823529411764706
k = 5: Akurasi = 0.8676470588235294
k = 6: Akurasi = 0.9117647058823529
k = 7: Akurasi = 0.9117647058823529
k = 8: Akurasi = 0.9117647058823529
k = 9: Akurasi = 0.9117647058823529
k = 10: Akurasi = 0.9117647058823529
k = 11: Akurasi = 0.8970588235294118
k = 12: Akurasi = 0.8970588235294118
k = 13: Akurasi = 0.8970588235294118
k = 14: Akurasi = 0.8970588235294118
k terbaik: 6 dengan akurasi: 0.9117647058823529
```

Terlihat bahwa akurasi meningkat seiring dengan peningkatan nilai k hingga mencapai puncaknya pada k = 6 hingga k = 10, di mana akurasi mencapai 91.18%. Setelah k = 10, akurasi mulai menurun kembali saat nilai k terus bertambah. Hal ini menunjukkan bahwa untuk dataset dan kasus yang diberikan, k = 6 memberikan keseimbangan yang baik antara underfitting dan overfitting.



```
1 # Mencetak prediksi berdasarkan k terbaik
2 classifier = KNNClassifier(k=6) # Membuat instance dari classifier
3 classifier.fit(X_train, y_train) # Melatih model
4 y_pred = classifier.predict(X_test) # Memprediksi data uji
5 y_pred # Mencetak hasil prediksi
```

Mencetak prediksi berdasarkan k terbaik dengan memprediksi data uji dari classifier.

```
[0,
 0,
 7,
 0,
 5,
 0,
 4,
 6,
 7,
 0,
 1,
 0,
 0,
 0,
 0,
 1,
 1,
 1,
 0,
```



EVALUASI

Evaluasi pada percobaan model ini dengan mendapatkan precision, recall, F1-Score, accuracy, dan confusion matrix pada setiap label kategori dan menampilkan banyaknya jumlah setiap label dari y_pred.

```
1 #evaluasi model
2 from collections import Counter
3
4 # Fungsi untuk membuat classification report
5 def classification_report(y_true, y_pred, labels):
6     report = {} # Inisialisasi dictionary kosong untuk menyimpan laporan
7     for label in labels:
8         true_positives = sum((true_label == label and pred_label == label) for true_label, pred_label in zip(y_true, y_pred)) # Menghitung true positives
9         false_positives = sum((true_label != label and pred_label == label) for true_label, pred_label in zip(y_true, y_pred)) # Menghitung false positives
10        false_negatives = sum((true_label == label and pred_label != label) for true_label, pred_label in zip(y_true, y_pred)) # Menghitung false negatives
11
12        precision = true_positives / (true_positives + false_positives) if (true_positives + false_positives) != 0 else 0 # Menghitung precision
13        recall = true_positives / (true_positives + false_negatives) if (true_positives + false_negatives) != 0 else 0 # Menghitung recall
14        f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) != 0 else 0 # Menghitung F1-score
15        accuracy = sum((true_label == pred_label) for true_label, pred_label in zip(y_true, y_pred)) / len(y_true) # Menghitung akurasi
16
17        report[label] = {'precision': precision, 'recall': recall, 'f1-score': f1_score, 'accuracy': accuracy} # Menyimpan metrik yang dihitung ke dalam dictionary report
18    return report # Mengembalikan laporan
```

Klasifikasi Data

```
20 # Fungsi untuk membuat confusion matrix
21 def confusion_matrix(y_true, y_pred, labels):
22     matrix = {} # Inisialisasi dictionary kosong untuk menyimpan matriks
23     for true_label in labels:
24         matrix[true_label] = {} # Inisialisasi dictionary kosong untuk setiap label benar
25         for pred_label in labels:
26             matrix[true_label][pred_label] = sum((true == true_label and pred == pred_label) for true, pred in zip(y_true, y_pred)) # Menghitung jumlah kejadian
27     return matrix # Mengembalikan matriks
```

```
29 # Tampilkan classification report
30 labels = list(set(y_test)) # Mendapatkan label unik dari dataset uji
31 report = classification_report(y_test, y_pred, labels) # Membuat laporan klasifikasi
32 print("Classification Report:") # Mencetak judul laporan klasifikasi
33 for label, metrics in report.items(): # Iterasi setiap label dan metriknya
34     print(f"Label: {label}")
35     print(f"    Precision: {metrics['precision']}")
36     print(f"    Recall: {metrics['recall']}")
37     print(f"    F1-Score: {metrics['f1-score']}")
38     print(f"    Accuracy: {metrics['accuracy']}")
```



```

40 # Tampilkan confusion matrix
41 confusion = confusion_matrix(y_test, y_pred, labels) # Membuat confusion matrix
42 print("Confusion Matrix:") # Mencetak judul confusion matrix
43 print("      ", end=" ") # Mencetak header untuk confusion matrix
44 for label in labels:
45     print(f"{label:5}", end=" ") # Mencetak setiap label
46 print()
47 for true_label in labels: # Iterasi setiap baris dari confusion matrix
48     print(f"{true_label:5}", end=" ")
49     for pred_label in labels:
50         print(f"{confusion[true_label][pred_label]:5}", end=" ") # Mencetak setiap sel dari confusion matrix
51     print() # Baris baru setelah setiap baris dari confusion matrix

```

```

➡ Classification Report:
Label: 0
  Precision: 0.9142857142857143
  Recall: 1.0
  F1-Score: 0.955223880597015
  Accuracy: 0.9117647058823529
Label: 1
  Precision: 0.8461538461538461
  Recall: 0.8461538461538461
  F1-Score: 0.8461538461538461
  Accuracy: 0.9117647058823529
Label: 3
  Precision: 0
  Recall: 0.0
  F1-Score: 0
  Accuracy: 0.9117647058823529
Label: 4
  Precision: 1.0
  Recall: 0.75
  F1-Score: 0.8571428571428571
  Accuracy: 0.9117647058823529
Label: 5
  Precision: 1.0
  Recall: 1.0
  F1-Score: 1.0
  Accuracy: 0.9117647058823529

```

```

Label: 6
  Precision: 1.0
  Recall: 1.0
  F1-Score: 1.0
  Accuracy: 0.9117647058823529
Label: 7
  Precision: 0.9
  Recall: 0.8181818181818182
  F1-Score: 0.8571428571428572
  Accuracy: 0.9117647058823529
Confusion Matrix:

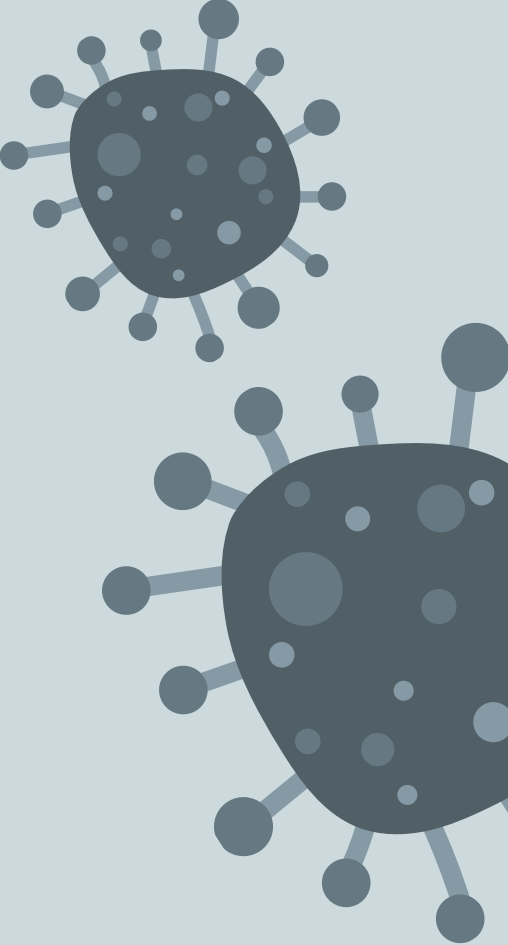
```

	0	1	3	4	5	6	7
0	32	0	0	0	0	0	0
1	2	11	0	0	0	0	0
3	0	0	0	0	0	0	1
4	0	1	0	3	0	0	0
5	0	0	0	0	6	0	0
6	0	0	0	0	0	1	0
7	1	1	0	0	0	0	9



```
1 # Menghitung jumlah setiap label dari y_pred
2 unique_labels, label_counts = np.unique(y_pred, return_counts=True)
3
4 # Mencetak hasil
5 print("Banyaknya jumlah setiap label dari y_pred:")
6 for label, count in zip(unique_labels, label_counts):
7     print(f"Label {label}: {count} instance") # Mencetak jumlah instance dari setiap label
```

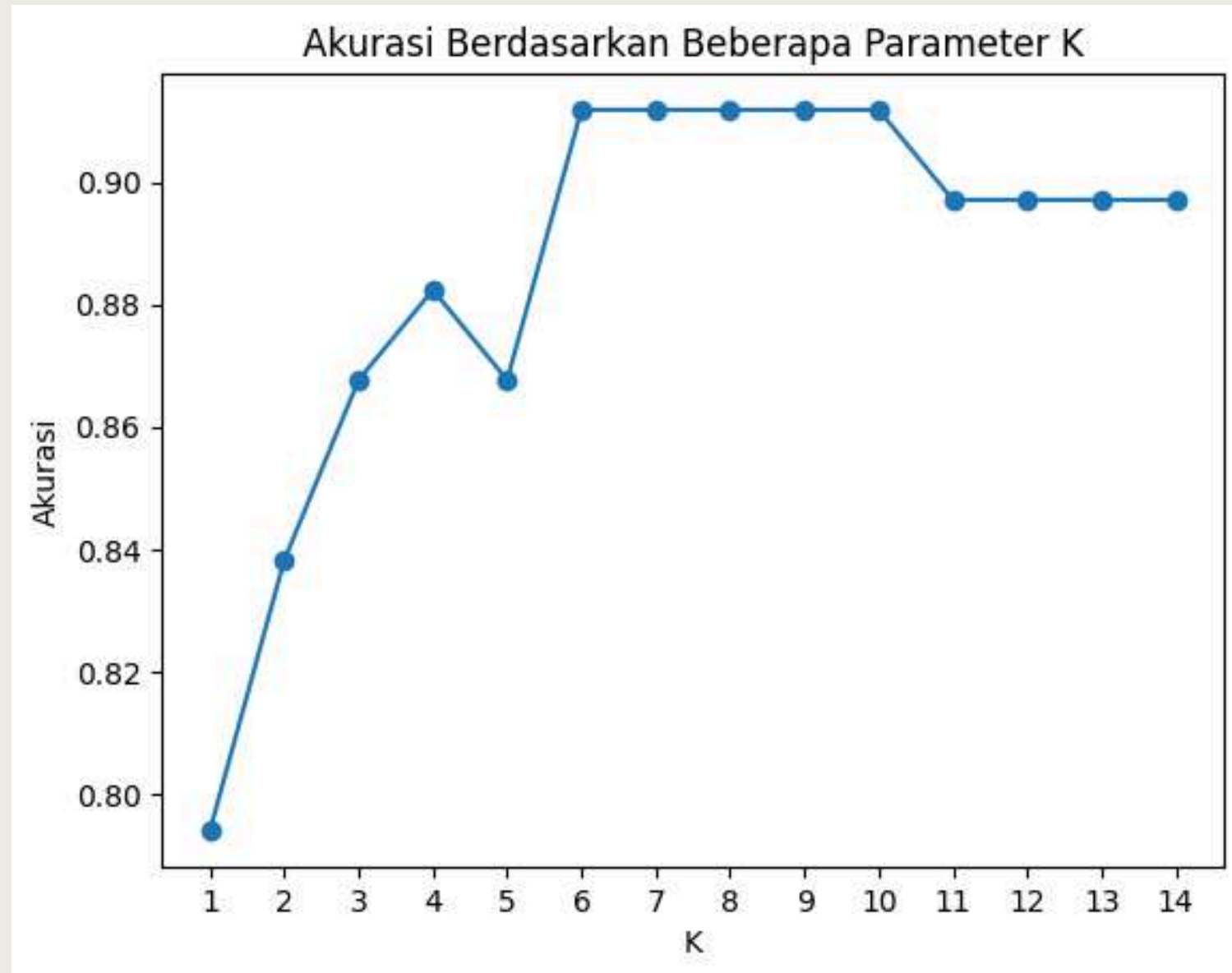
```
[>] Banyaknya jumlah setiap label dari y_pred:
Label 0: 35 instance
Label 1: 13 instance
Label 4: 3 instance
Label 5: 6 instance
Label 6: 1 instance
Label 7: 10 instance
```



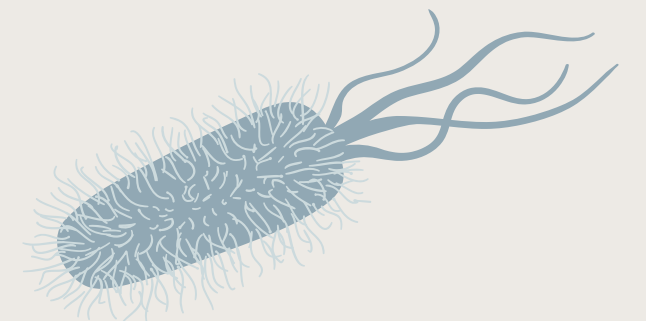


HASIL & ANALISIS

Klasifikasi Data



Visualisasi Grafik Akurasi. terlihat bahwa akurasi meningkat seiring dengan peningkatan nilai k hingga mencapai puncaknya pada $k=13$, di mana akurasi mencapai 91.18%. Setelah $k = 13$, akurasi mulai menurun kembali saat nilai k terus bertambah. Hal ini menunjukkan bahwa untuk dataset dan kasus yang diberikan, $k = 13$ memberikan keseimbangan yang baik antara underfitting dan overfitting. Pada saat nilai k terlalu rendah, model cenderung overfitting karena terlalu sensitif terhadap noise dalam data, sedangkan saat nilai k terlalu tinggi, model menjadi terlalu sederhana dan cenderung underfitting.



KESIMPULAN

Dari hasil analisis yang dilakukan terhadap model klasifikasi k-Nearest Neighbors (kNN) pada dataset ecoli, dapat disimpulkan beberapa hal.

Akurasi pada Model k-NN menunjukkan performa yang baik dalam melakukan klasifikasi pada dataset yang diberikan. Dengan menggunakan k-NN dan menentukan parameter k yang sesuai, akurasi model dapat mencapai nilai yang tinggi, dalam kasus ini antara 0.79 hingga 0.91.

Hasil menunjukkan bahwa pemilihan nilai k yang tepat memiliki dampak yang signifikan terhadap kinerja model. Nilai k yang tidak tepat dapat menyebabkan overfitting atau underfitting. Oleh karena itu, penting untuk melakukan eksperimen dan validasi untuk menentukan nilai k yang optimal.

Hasil prediksi menunjukkan variasi dalam persebaran kelas dari hasil prediksi. Hal ini menunjukkan bahwa model dapat mengidentifikasi kelas-kelas yang berbeda dengan baik, meskipun ada perbedaan dalam jumlah instance antara kelas-kelas tersebut.

Visualisasi grafik akurasi menunjukkan tren peningkatan akurasi seiring dengan peningkatan nilai k hingga mencapai puncaknya pada $k=6$. Setelah itu, akurasi mulai menurun seiring dengan peningkatan nilai k. Hal ini menunjukkan bahwa penggunaan nilai k yang terlalu kecil atau terlalu besar dapat menyebabkan overfitting atau underfitting.